

User Management API

▼ Relevant source files

Backend/STUDER/src/main/java/facu/studer/controllers/UserController.java

Backend/STUDER/src/main/java/facu/studer/entities/User.java

Backend/STUDER/src/main/java/facu/studer/exceptions/ApiError.java

Backend/STUDER/src/main/java/facu/studer/exceptions/GlobalExceptionHandler.java

Backend/STUDER/src/main/java/facu/studer/mappers/UserMapper.java

Backend/STUDER/src/main/java/facu/studer/repositories/UserRepository.java

Backend/STUDER/src/main/java/facu/studer/services/UserService.java

Backend/STUDER/src/main/java/facu/studer/services/implementation/UserServiceImpl.java

The User Management API provides the essential infrastructure for managing identity, profiles, and discovery within the STUDER platform. It handles the lifecycle of a user account—from registration and profile updates to soft deletion—and manages complex profile picture processing via integration with the media service.

API Endpoints

The `UserController` exposes endpoints under the `/api/v1/users` prefix. While registration is public, most operations require a valid JWT and enforce ownership checks, ensuring users can only modify their own data.

Method	Endpoint	Description	Auth Required
POST	<code>/register</code>	Registers a new user account.	No
PUT	<code>/</code>	Updates authenticated user profile (supports multipart/form-data).	Yes
DELETE	<code>/</code>	Soft deletes the authenticated user account.	Yes
GET	<code>/me</code>	Returns full profile data for the current user.	Yes

Method	Endpoint	Description	Auth Required
GET	/username/{username}	Returns public profile data for a specific user.	Yes
GET	/search	Paginated search for users by username.	Yes
GET	/id	Gets user data by ID (restricted to self).	Yes

Sources: Backend/STUDER/src/main/java/facu/studer/controllers/UserController.java 24-97

Business Logic & Implementation

The `UserServiceImpl` class implements the core business logic, including data validation, password hashing, and interaction with external services.

Data Flow: User Update & Image Processing

When a user updates their profile with a new image, the backend acts as an orchestrator between the client and the Python-based Media Service.

- Request Reception:** `UserController` receives a `MultipartFile` and a `UserUpdateRequestDTO`
Backend/STUDER/src/main/java/facu/studer/controllers/UserController.java 47-51
- Media Service Call:** `UserServiceImpl` forwards the file to the Media Service at `${app.media.service.url}/upload-image/profile` using `RestTemplate`
Backend/STUDER/src/main/java/facu/studer/services/implementation/UserServiceImpl.java 91-103
- Variant Storage:** The Media Service returns a map of URLs for different image variants. The `User` entity stores these in four distinct fields
Backend/STUDER/src/main/java/facu/studer/entities/User.java 70-81 :
 - `profilePictureOriginalUrl` : The raw uploaded file.
 - `profilePictureAvatarUrl` : 1080p high-quality version.
 - `profilePictureWebpUrl` : Optimized WebP format for performance.
 - `profilePictureThumbnailUrl` : 150x150 small crop for lists.
- Persistence:** The `UserRepository` saves the updated `User` entity
Backend/STUDER/src/main/java/facu/studer/services/implementation/UserServiceImpl.java 120-121

Soft Delete Pattern

Instead of removing rows from the database, STUDER uses a soft-delete mechanism. The

`softDelete` method sets the `isActive` flag (inherited from `BaseEntity`) to `false`

Backend/STUDER/src/main/java/facu/studer/services/implementation/UserServiceImpl.java 132-134

This preserves data integrity for related entities like discussions or messages while preventing the user from logging in.

Sources:

Backend/STUDER/src/main/java/facu/studer/services/implementation/UserServiceImpl.java 32-136

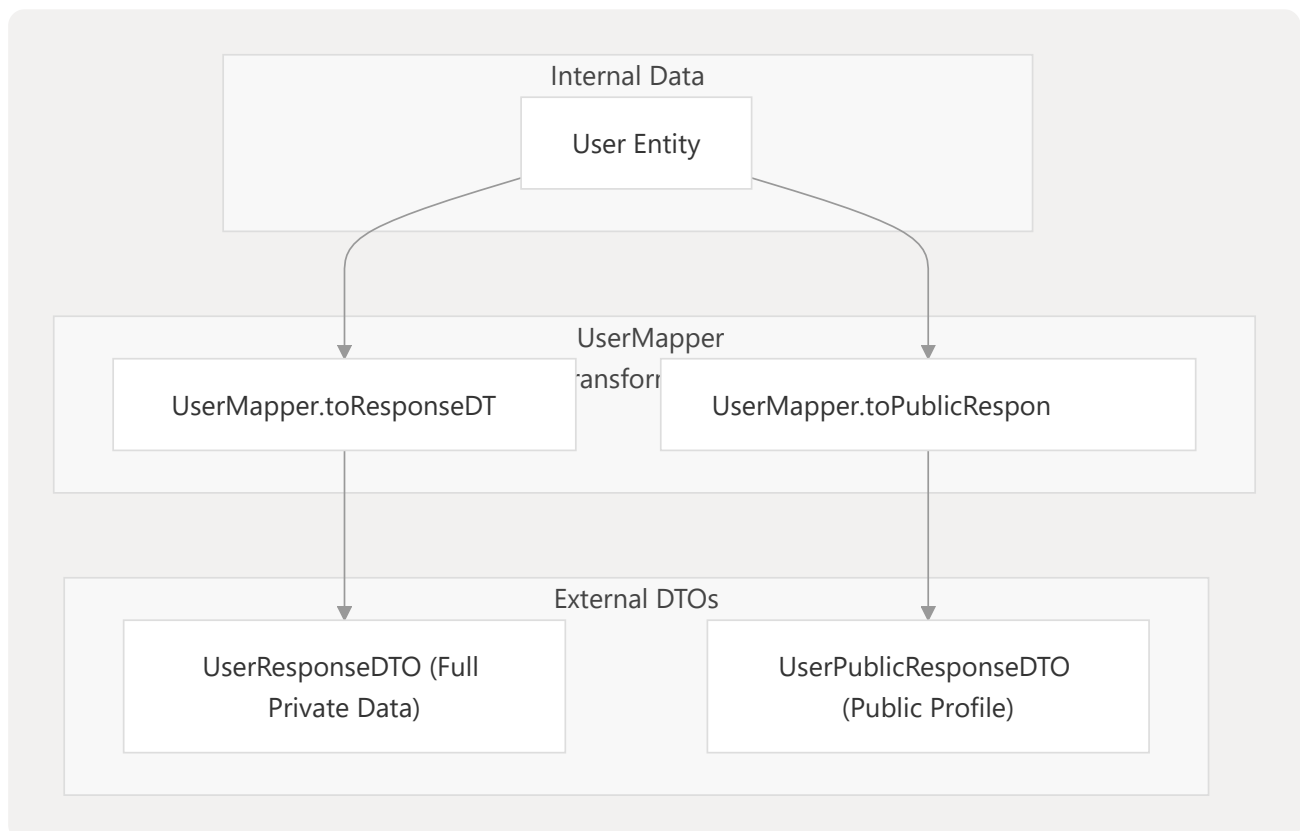
Backend/STUDER/src/main/java/facu/studer/entities/User.java 26-82

Data Transformation (Mappers)

The `UserMapper` utility class handles the conversion between internal `User` entities and various Data Transfer Objects (DTOs) to ensure sensitive information (like hashed passwords) is never leaked to the frontend.

Mapping Architecture

Title: User Entity to DTO Mapping Flow



Sources: Backend/STUDER/src/main/java/facu/studer/mappers/UserMapper.java 11-76

Error Handling Contract

The API uses a centralized `GlobalExceptionHandler` to catch exceptions and return a standardized `ApiError` response. This contract ensures the frontend can consistently handle validation errors and internationalized messages.

The ApiError Structure

The `ApiError` class Backend/STUDER/src/main/java/facu/studer/exceptions/ApiError.java 17-47

includes:

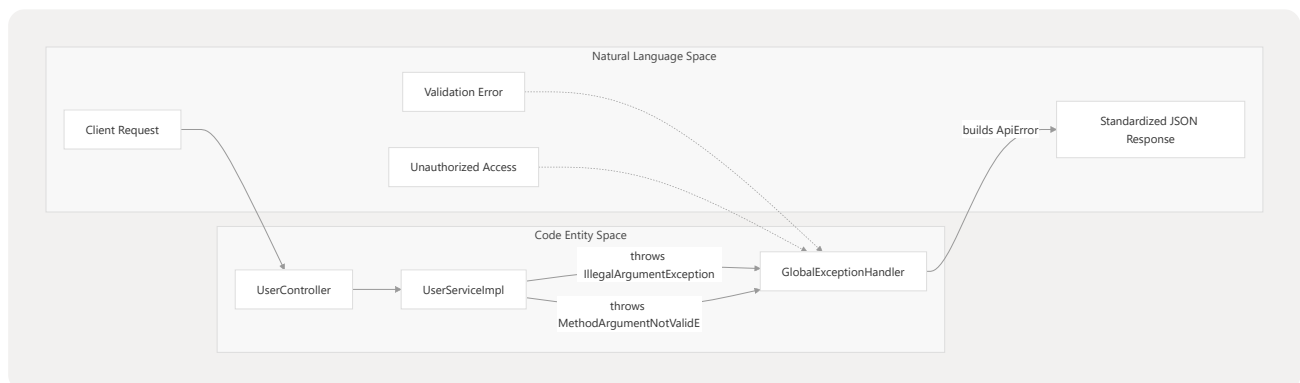
- `status` : HTTP status code.
- `code` : A machine-readable string (e.g., `user.email.unique`) used as an i18n key.
- `message` : A human-readable, localized message resolved via `MessageSource`
- `fieldErrors` : A map specifically for `MethodArgumentNotValidException`, linking specific fields (e.g., "email") to their validation failure reasons

Backend/STUDER/src/main/java/facu/studer/exceptions/GlobalExceptionHandler.java 93-104

Backend/STUDER/src/main/java/facu/studer/exceptions/GlobalExceptionHandler.java 31-53

Exception Handling Logic

Title: Exception Resolution Process



Sources:

Backend/STUDER/src/main/java/facu/studer/exceptions/GlobalExceptionHandler.java 21-107

Backend/STUDER/src/main/java/facu/studer/exceptions/ApiError.java 14-47

Persistence Layer

The `UserRepository` interface extends `JpaRepository` and provides specific query methods for user lookup and uniqueness validation.

- **Uniqueness Checks:** `existsByEmail` and `existsByUsername` are used during registration and updates to prevent collisions

Backend/STUDER/src/main/java/facu/studer/repositories/UserRepository.java 20-27

- **Search:** `findByUsernameContainingIgnoreCase` supports the user discovery feature with paginated results

Backend/STUDER/src/main/java/facu/studer/repositories/UserRepository.java 49-50

Sources: Backend/STUDER/src/main/java/facu/studer/repositories/UserRepository.java 14-50